

Lustre to Multi-cores via Trampoline

Implementation of Synchronous Data-Flow Programs on a Dual-Core Architecture using a Real-Time Operating System (RTOS)

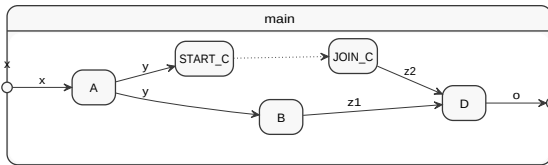
Florian Perret^{1,*}

¹ Univ. Grenoble Alpes, VERIMAG, 38000 Grenoble, France

* E-mail: florian.perret@etu.univ-grenoble-alpes.fr

1 Synchronous Data-Flow & Lustre

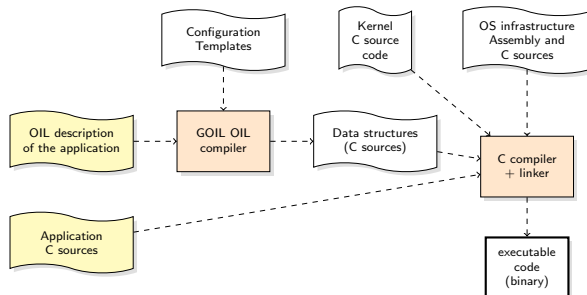
- **The Synchronous Approach:** Network of deterministic nodes. It offers strict determinism and bounded memory, providing perfect safety guarantees for **single-core** execution.
- **The Compilation Paradox:** Lustre is inherently parallel, but classical compilation flattens it into sequential C code. Our goal is to restore this concurrency for **multi-core** targets.
- **Parallelism Extraction (lv6):** Restores concurrency by splitting node execution into *Start* and *Join* primitives.



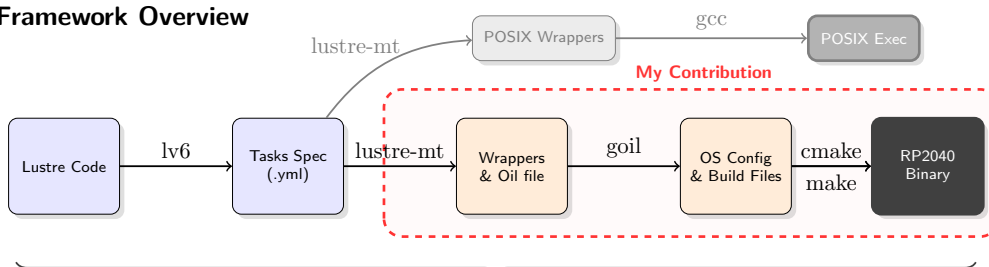
2 Trampoline RTOS & Hardware

- **Raspberry Pi Pico (Cortex-M0+):**
 - Micro-controller class dual-core architecture.
 - Hardware FIFOs and Spinlocks for inter-core communication and synchronization.

- **Trampoline Compilation Chain:**



4 Framework Overview



Automation: The script generated by the lv6 compiler for POSIX was extended to also automate this entire pipeline for the Raspberry Pi Pico.

To enable this port, the Trampoline codebase was modified and extended, primarily within the **Configuration Templates** and **OS Infrastructure**:

- **Multi-Core RTOS Port (OS & Templates modifications):**
 - **Boot & Memory:** Core 1 stack allocation via linker script, and pre-OS stack isolation (MSP/PSP spacing) to support early multicore system calls.
 - **Kernel Accesses:** Adapted low-level handlers to dynamically access existing OS array structures based on the Core ID.
 - **Synchronization:** Integration of hardware Spinlocks and standard interrupt disabling into OS handlers to protect concurrent kernel accesses.
 - **Communication:** Inter-Core Interrupts via FIFOs to manage cross-core task activations.

3 Automated Code Generation

A custom generator (*lustre-mt*) is extended to output:

- **Lustre Wrappers:**
 - *Synchronisation:* Implements *Start/Join* primitives using OS events (*WaitEvent*, *SetEvent*).
 - *Memory:* Secures data exchange via Inter-OS-Application Communication (*IOC*).
- **Static OS Configuration:**
 - Defines cores, alarms, tasks, and communication routing.
- *Timing:* Maps the Lustre discrete time step to a periodic OS Alarm.
- **C Templates:**
 - Generates the OS startup sequence and user stubs for custom I/Os and hardware initialization.

5 Future Work

- **Temporal Validation:** WCET tools & **MIA** (Multi-core Interference Analysis).
- **Smart Allocation:** Automated task partitioning.
- **Portability:** Support for other multi-core architectures.